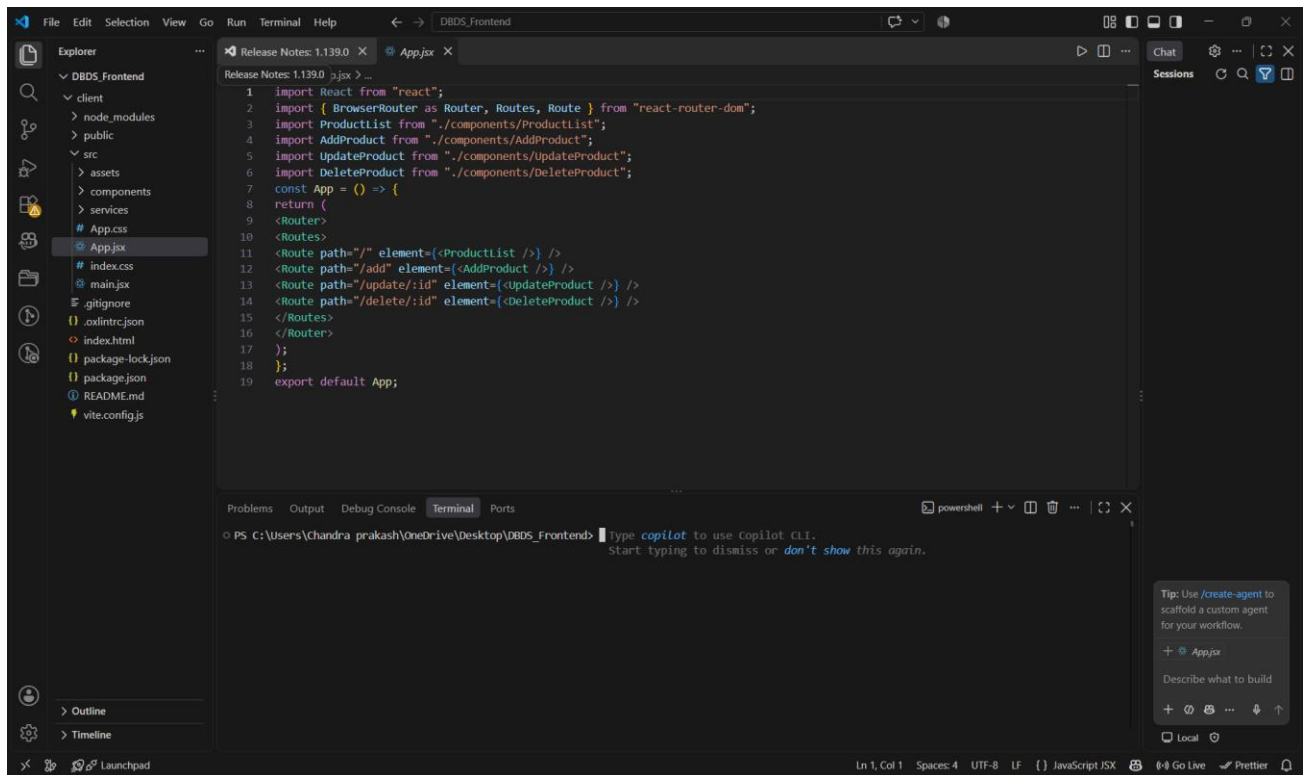
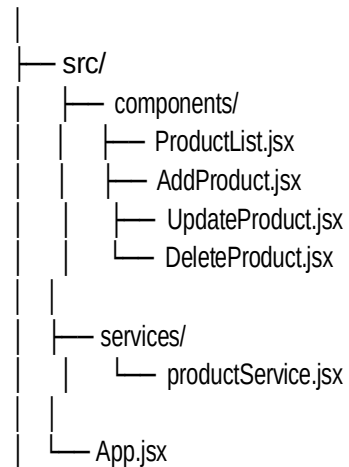


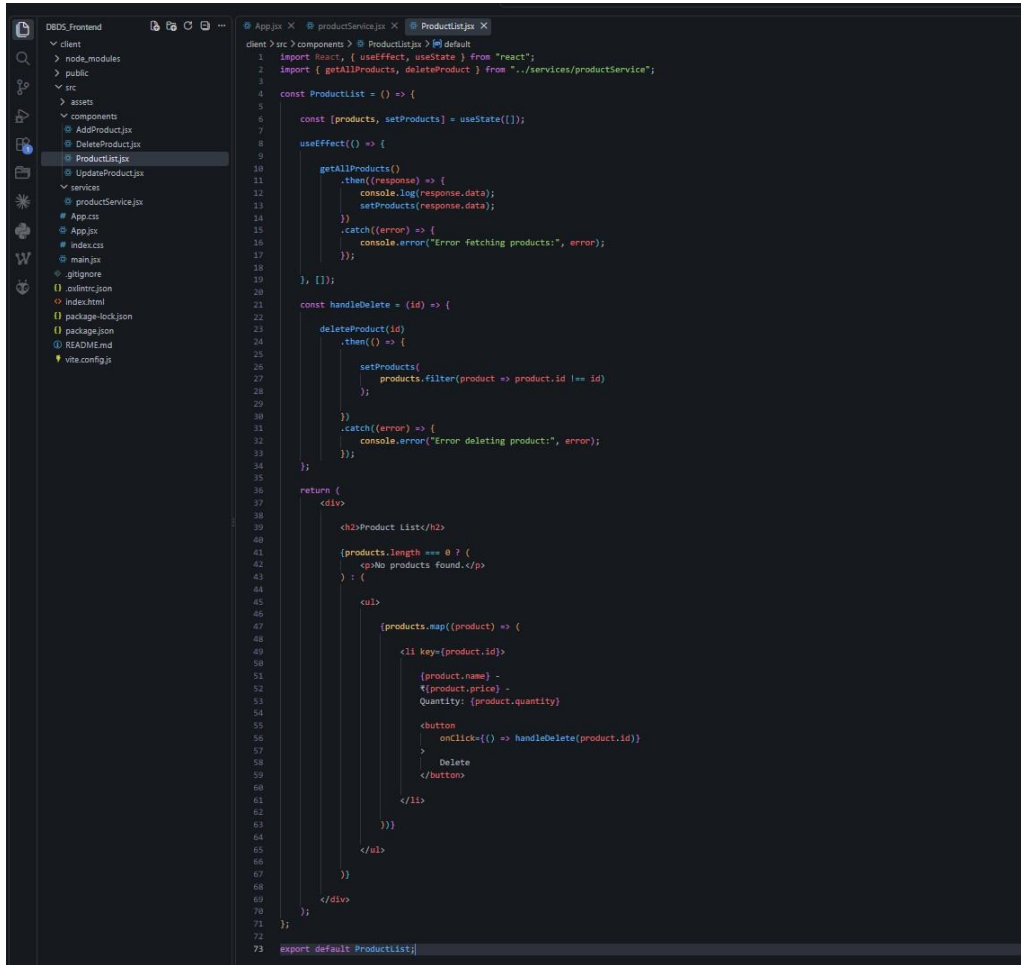
REACTJS + SPRING BOOT CRUD

frontend/



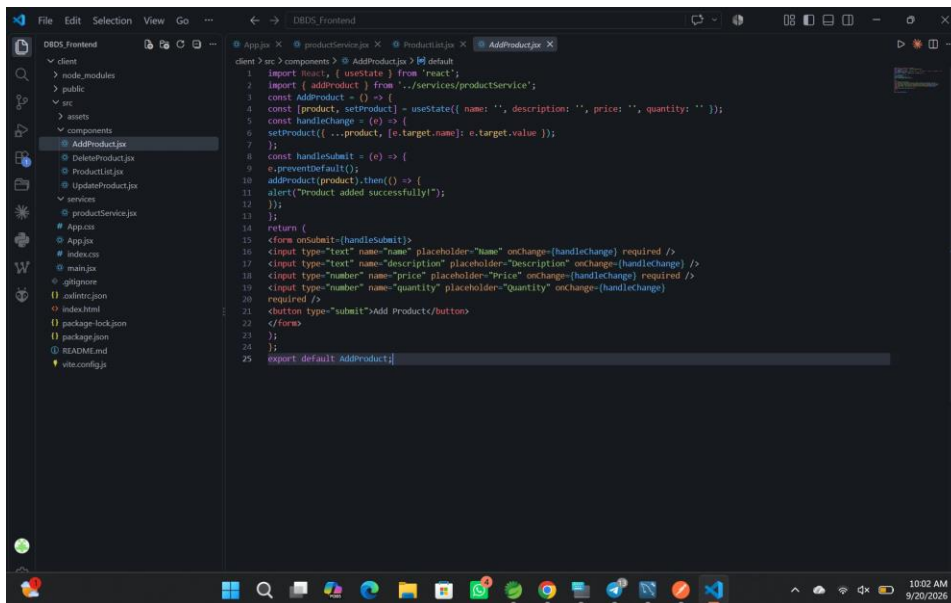
ProductList.jsx

REACTJS + SPRING BOOT CRUD



```
client > src > components > ProductList.jsx > @ default
1 import React, { useEffect, useState } from "react";
2 import { getAllProducts, deleteProduct } from "../services/productService";
3
4 const ProductList = () => {
5
6   const [products, setProducts] = useState([]);
7
8   useEffect(() => {
9
10     getAllProducts()
11       .then((response) => {
12         console.log(response.data);
13         setProducts(response.data);
14       })
15       .catch((error) => {
16         console.error("Error fetching products:", error);
17       });
18   });
19
20   const handleDelete = (id) => {
21     deleteProduct(id)
22       .then(() => {
23         setProducts(
24           products.filter(product => product.id !== id)
25         );
26       })
27       .catch((error) => {
28         console.error("Error deleting product:", error);
29       });
30   });
31
32   return (
33     <div>
34       <h2>Product List</h2>
35
36       (products.length === 0 ? (
37         <p>No products found.</p>
38       ) : (
39         <ul>
40           {products.map((product) => (
41             <li key={product.id}>
42               {product.name} -
43               {product.price} -
44               Quantity: {product.quantity}
45
46               <button
47                 onClick={() => handleDelete(product.id)}
48               >
49                 Delete
50               </button>
51             </li>
52           ))}
53         </ul>
54       )}
55     </div>
56   );
57 };
58
59 export default ProductList;
```

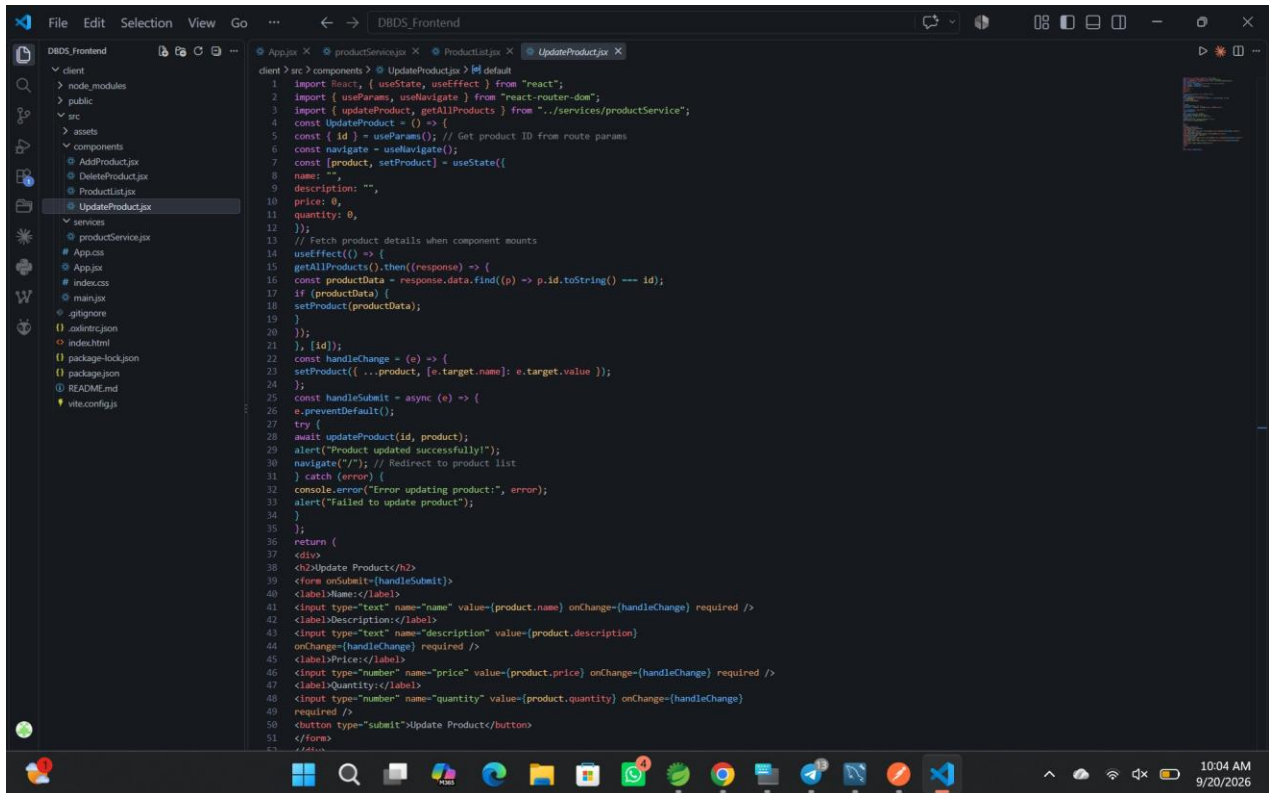
AddProduct.jsx



```
client > src > components > AddProduct.jsx > @ default
1 import React, { useState } from "react";
2 import { addProduct } from "../services/productService";
3
4 const AddProduct = () => {
5   const [product, setProduct] = useState({ name: "", description: "", price: "", quantity: "" });
6   const handleChange = (e) => {
7     setProduct({ ...product, [e.target.name]: e.target.value });
8   };
9
10   const handleSubmit = (e) => {
11     e.preventDefault();
12     addProduct(product).then(() => {
13       alert("Product added successfully!");
14     });
15   };
16
17   return (
18     <form onSubmit={handleSubmit}>
19       <input type="text" name="name" placeholder="Name" onChange={handleChange} required />
20       <input type="text" name="description" placeholder="Description" onChange={handleChange} />
21       <input type="number" name="price" placeholder="Price" onChange={handleChange} required />
22       <input type="number" name="quantity" placeholder="Quantity" onChange={handleChange} required />
23       <button type="submit">Add Product</button>
24     </form>
25   );
26 };
27
28 export default AddProduct;
```

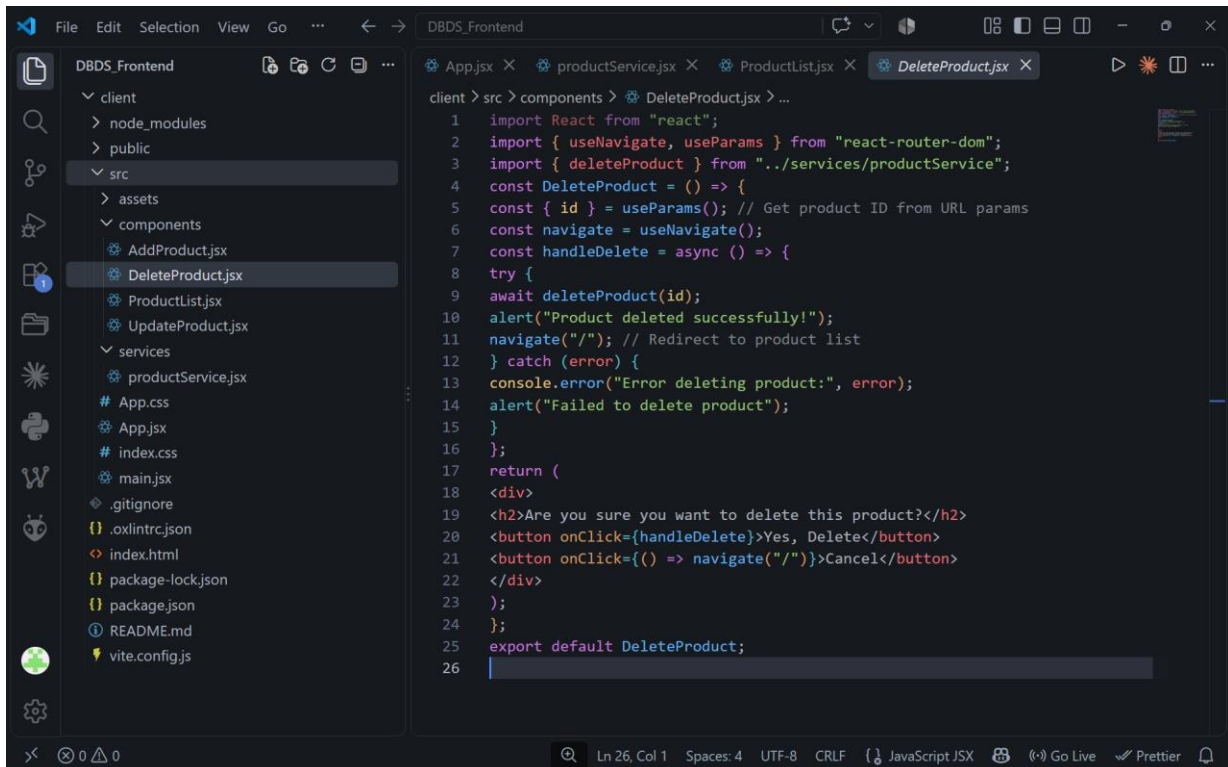
REACTJS + SPRING BOOT CRUD

UpdateProduct.jsx



```
client > src > components > UpdateProduct.jsx > default
1 import React, { useState, useEffect } from "react";
2 import { useNavigate, useParams } from "react-router-dom";
3 import { deleteProduct, getAllProducts } from "../services/productService";
4 const DeleteProduct = () => {
5   const { id } = useParams(); // Get product ID from route params
6   const navigate = useNavigate();
7   const [product, setProduct] = useState({
8     name: "",
9     description: "",
10    price: 0,
11    quantity: 0,
12  });
13  // Fetch product details when component mounts
14  useEffect(() => {
15    getAllProducts().then((response) => {
16      const productData = response.data.find((p) => p.id.toString() === id);
17      if (productData) {
18        setProduct(productData);
19      }
20    });
21  }, [id]);
22  const handleChange = (e) => {
23    setProduct({ ...product, [e.target.name]: e.target.value });
24  };
25  const handleSubmit = async (e) => {
26    e.preventDefault();
27    try {
28      await deleteProduct(id, product);
29      alert("Product deleted successfully!");
30      navigate("/"); // Redirect to product list
31    } catch (error) {
32      console.error("Error deleting product:", error);
33      alert("Failed to delete product");
34    }
35  };
36  return (
37    <div>
38      <h2>Delete Product</h2>
39      <form onSubmit={handleSubmit}>
40        <label>Name:</label>
41        <input type="text" name="name" value={product.name} onChange={handleChange} required />
42        <label>Description:</label>
43        <input type="text" name="description" value={product.description}
44          onChange={handleChange} required />
45        <label>Price:</label>
46        <input type="number" name="price" value={product.price} onChange={handleChange} required />
47        <label>Quantity:</label>
48        <input type="number" name="quantity" value={product.quantity} onChange={handleChange}
49          required />
50        <button type="submit">Delete Product</button>
51      </form>
52    </div>
53  );
54};
55export default DeleteProduct;
```

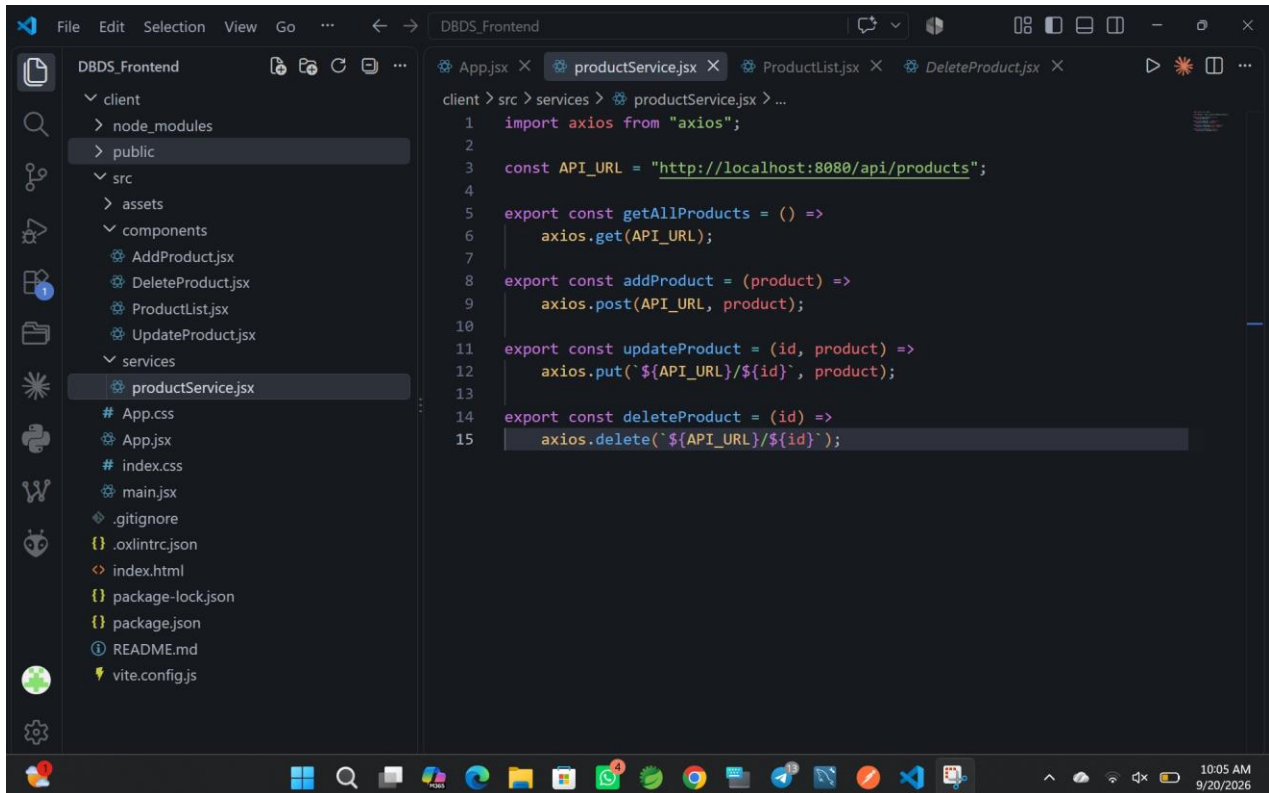
DeleteProduct.jsx



```
client > src > components > DeleteProduct.jsx > ...
1 import React from "react";
2 import { useNavigate, useParams } from "react-router-dom";
3 import { deleteProduct } from "../services/productService";
4 const DeleteProduct = () => {
5   const { id } = useParams(); // Get product ID from URL params
6   const navigate = useNavigate();
7   const handleDelete = async () => {
8     try {
9       await deleteProduct(id);
10      alert("Product deleted successfully!");
11      navigate("/"); // Redirect to product list
12    } catch (error) {
13      console.error("Error deleting product:", error);
14      alert("Failed to delete product");
15    }
16  };
17  return (
18    <div>
19      <h2>Are you sure you want to delete this product?</h2>
20      <button onClick={handleDelete}>Yes, Delete</button>
21      <button onClick={() => navigate("/")}>Cancel</button>
22    </div>
23  );
24};
25export default DeleteProduct;
26
```

REACTJS + SPRING BOOT CRUD

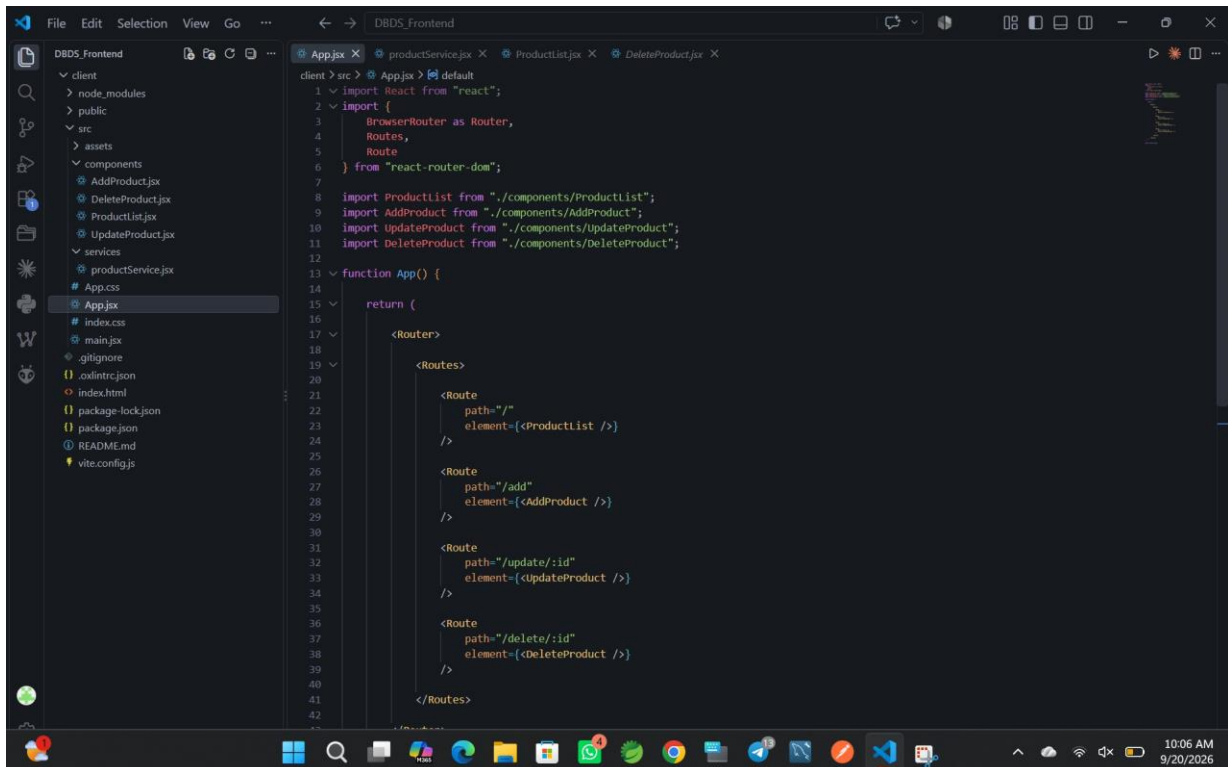
productService.jsx



The screenshot shows a VS Code editor window titled 'DBDS_Frontend'. The file explorer on the left shows the project structure with 'productService.jsx' selected under the 'services' directory. The main editor displays the code for 'productService.jsx'.

```
client > src > services > productService.jsx > ...
1  import axios from "axios";
2
3  const API_URL = "http://localhost:8080/api/products";
4
5  export const getAllProducts = () =>
6  {
7    axios.get(API_URL);
8  }
9
10 export const addProduct = (product) =>
11 {
12   axios.post(API_URL, product);
13 }
14
15 export const updateProduct = (id, product) =>
16 {
17   axios.put(`${API_URL}/${id}`, product);
18 }
19
20 export const deleteProduct = (id) =>
21 {
22   axios.delete(`${API_URL}/${id}`);
23 }
```

App.jsx:

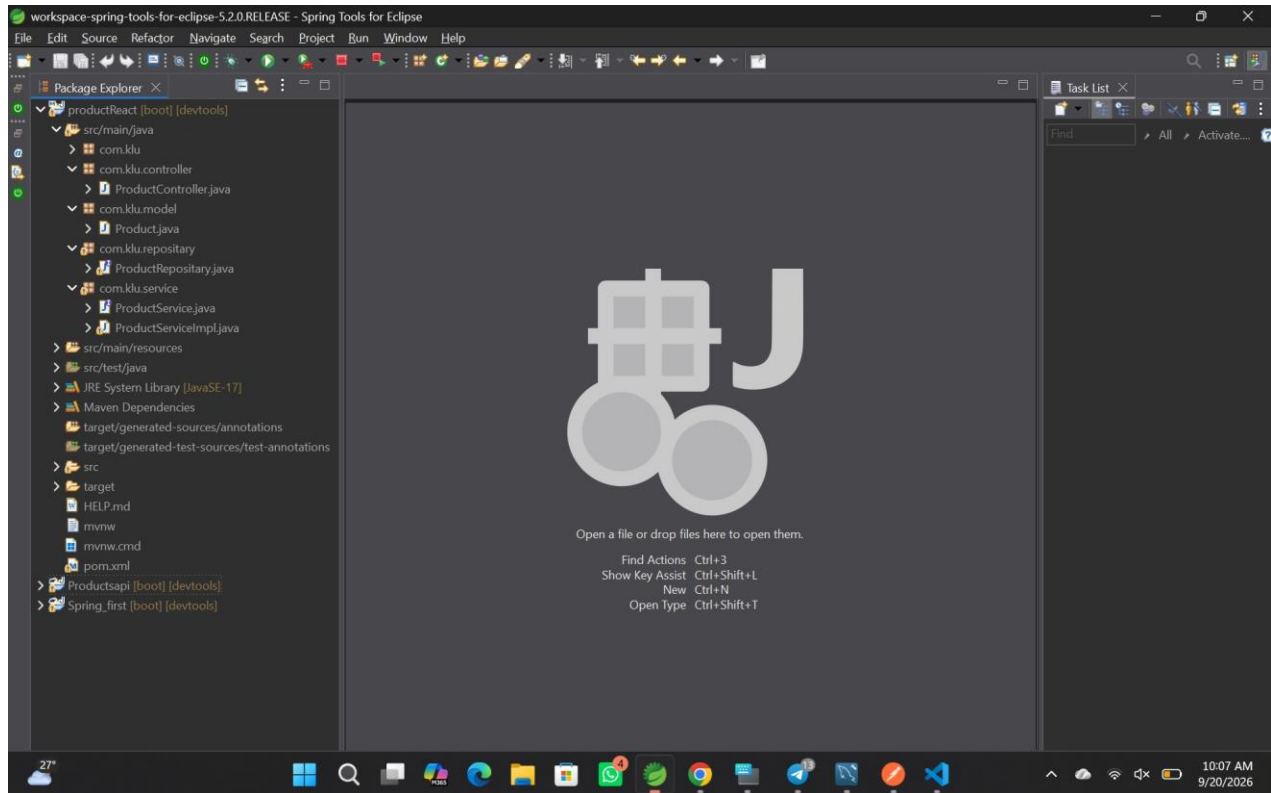


The screenshot shows a VS Code editor window titled 'DBDS_Frontend'. The file explorer on the left shows the project structure with 'App.jsx' selected under the 'components' directory. The main editor displays the code for 'App.jsx'.

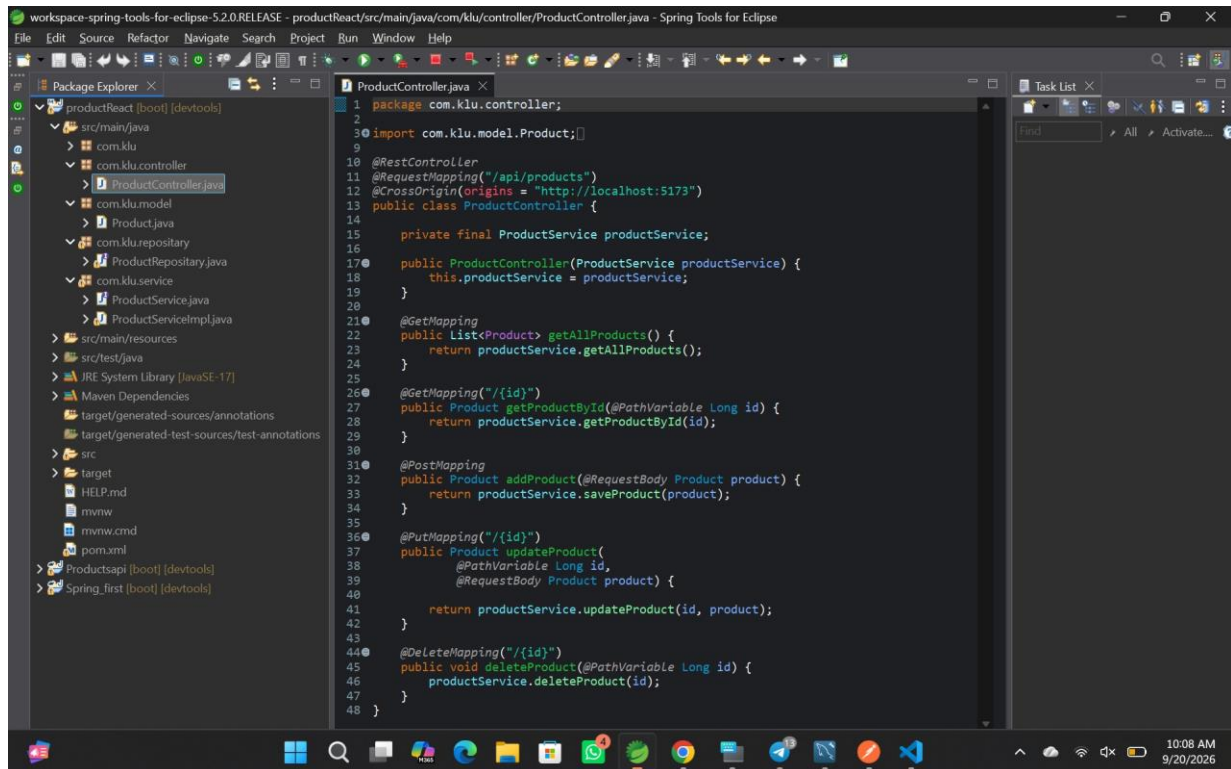
```
client > src > App.jsx > default
1  import React from "react";
2  import {
3    BrowserRouter as Router,
4    Routes,
5    Route
6  } from "react-router-dom";
7
8  import ProductList from "../components/ProductList";
9  import AddProduct from "../components/AddProduct";
10 import UpdateProduct from "../components/UpdateProduct";
11 import DeleteProduct from "../components/DeleteProduct";
12
13 function App() {
14
15   return (
16     <Router>
17       <Routes>
18         <Route
19           path="/"
20           element={<ProductList />}
21         />
22         <Route
23           path="/add"
24           element={<AddProduct />}
25         />
26         <Route
27           path="/update/:id"
28           element={<UpdateProduct />}
29         />
30         <Route
31           path="/delete/:id"
32           element={<DeleteProduct />}
33         />
34       </Routes>
35     </Router>
36   );
37 }
```

REACTJS + SPRING BOOT CRUD

Backend(SpringBoot)

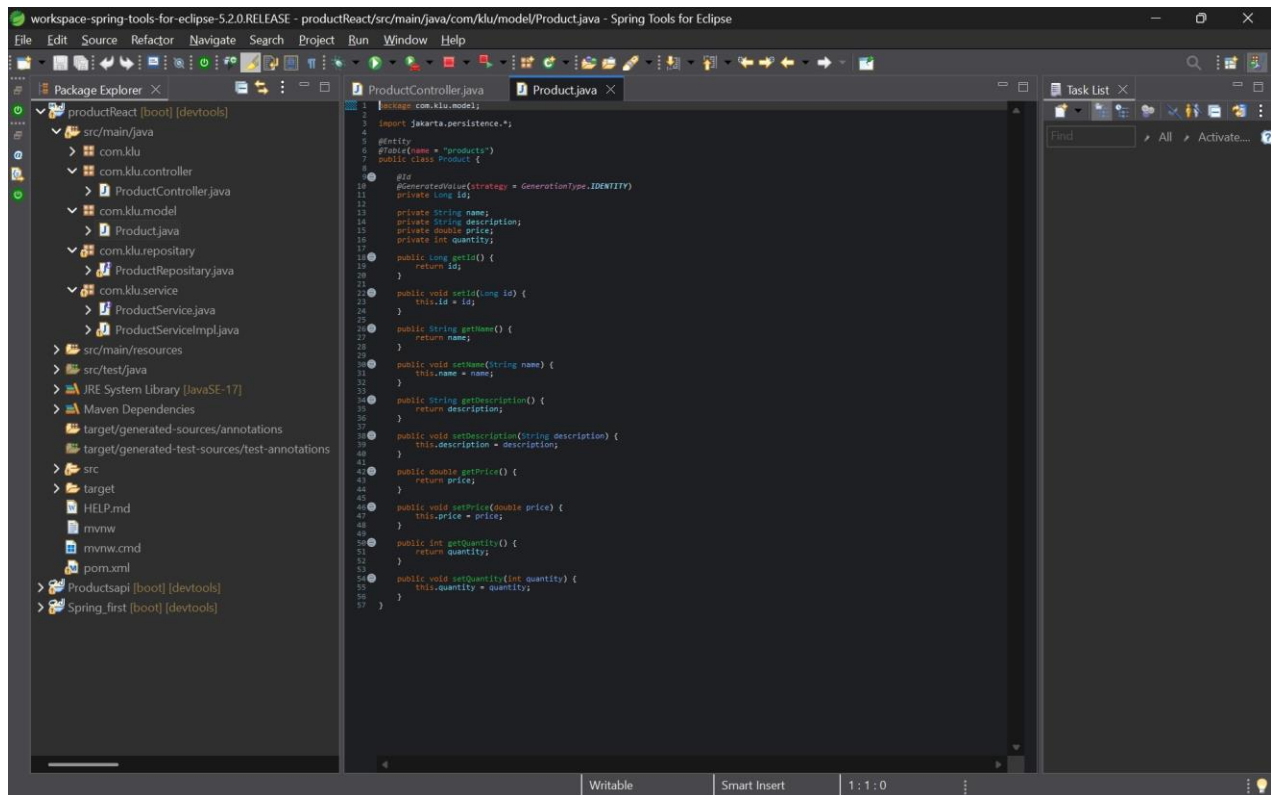


ProductController.java



REACTJS + SPRING BOOT CRUD

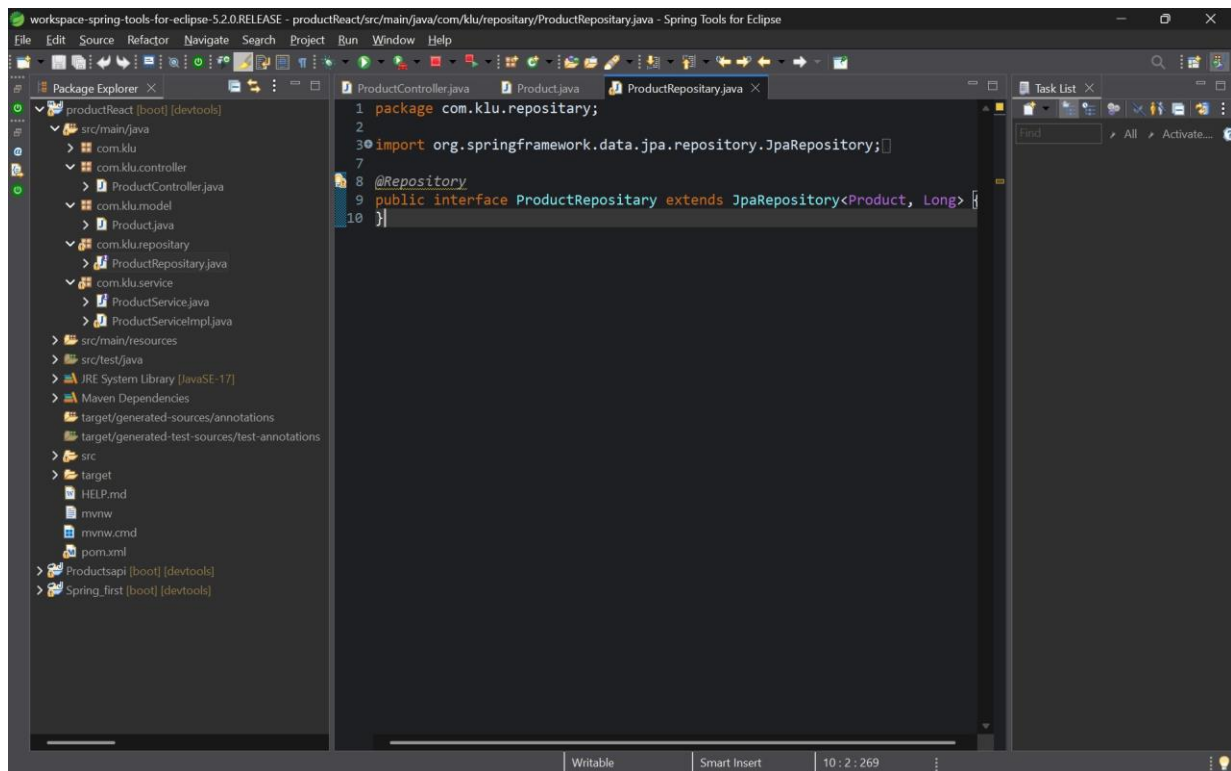
Product.java



The screenshot shows the Eclipse IDE with the 'Product.java' file open. The Package Explorer on the left shows the project structure: 'productReact' (boot) [devtools] contains 'src/main/java' which includes 'com.klu' (controller, model, repository, service, serviceimpl). The main editor displays the 'Product.java' code, which is a JPA entity class. The code includes package, imports, annotations, and methods for id, name, description, price, and quantity.

```
1 package com.klu.model;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "products")
7 public class Product {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    private String name;
14    private String description;
15    private double price;
16    private int quantity;
17
18    public Long getId() {
19        return id;
20    }
21
22    public void setId(Long id) {
23        this.id = id;
24    }
25
26    public String getName() {
27        return name;
28    }
29
30    public void setName(String name) {
31        this.name = name;
32    }
33
34    public String getDescription() {
35        return description;
36    }
37
38    public void setDescription(String description) {
39        this.description = description;
40    }
41
42    public double getPrice() {
43        return price;
44    }
45
46    public void setPrice(double price) {
47        this.price = price;
48    }
49
50    public int getQuantity() {
51        return quantity;
52    }
53
54    public void setQuantity(int quantity) {
55        this.quantity = quantity;
56    }
57 }
```

ProductRepository.java

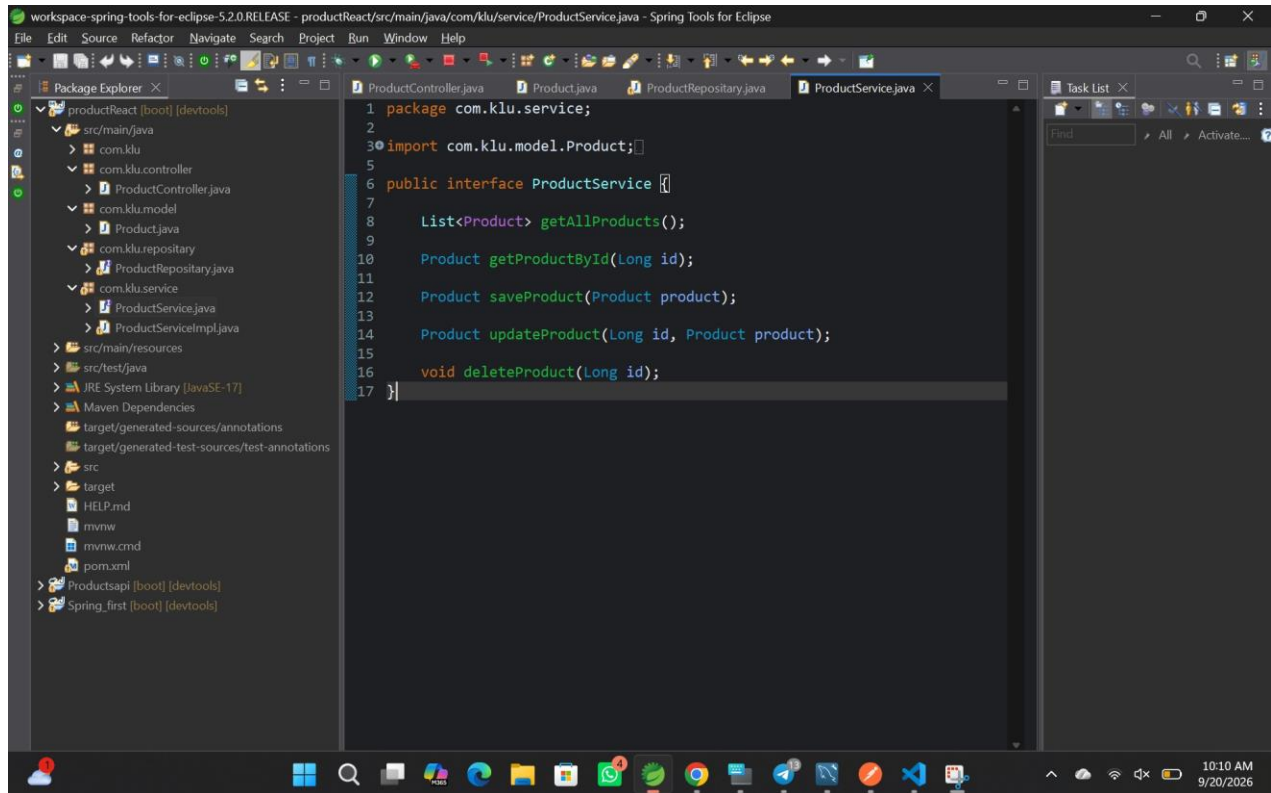


The screenshot shows the Eclipse IDE with the 'ProductRepository.java' file open. The Package Explorer on the left shows the project structure. The main editor displays the 'ProductRepository.java' code, which is a Spring Data JPA repository interface. The code includes package, imports, and the repository interface definition.

```
1 package com.klu.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 @Repository
6 public interface ProductRepository extends JpaRepository<Product, Long> {
7
8 }
9 }
```

REACTJS + SPRING BOOT CRUD

ProductService.java

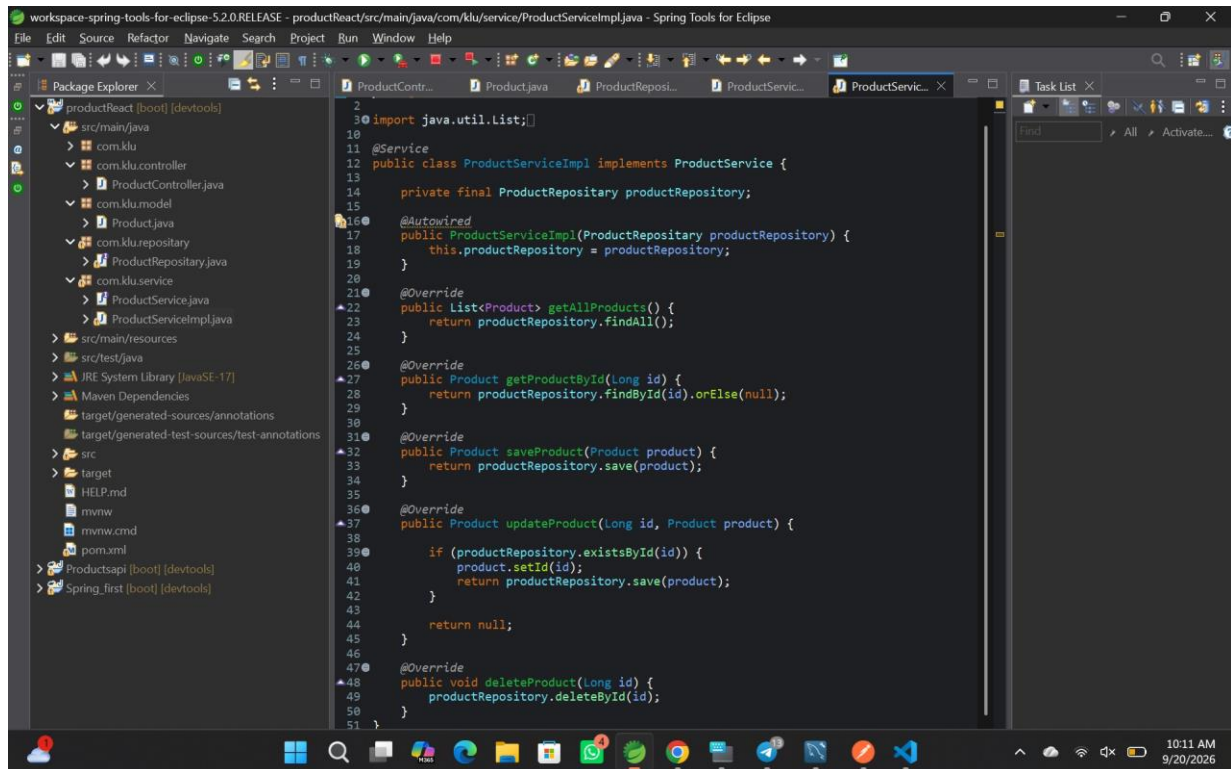


The screenshot shows the Eclipse IDE with the 'ProductService.java' file open. The Package Explorer on the left shows the project structure: 'productReact [boot] [devtools]' with sub-packages 'src/main/java' (containing 'com.klu' with 'controller', 'model', 'repository', and 'service' sub-packages) and 'src/test/java'. The main editor displays the following code:

```
1 package com.klu.service;
2
3 import com.klu.model.Product;
4
5
6 public interface ProductService {
7
8     List<Product> getAllProducts();
9
10    Product getProductById(Long id);
11
12    Product saveProduct(Product product);
13
14    Product updateProduct(Long id, Product product);
15
16    void deleteProduct(Long id);
17 }
```

The Task List on the right is empty. The status bar at the bottom shows the time as 10:10 AM on 9/20/2026.

ProductServiceImpl.java



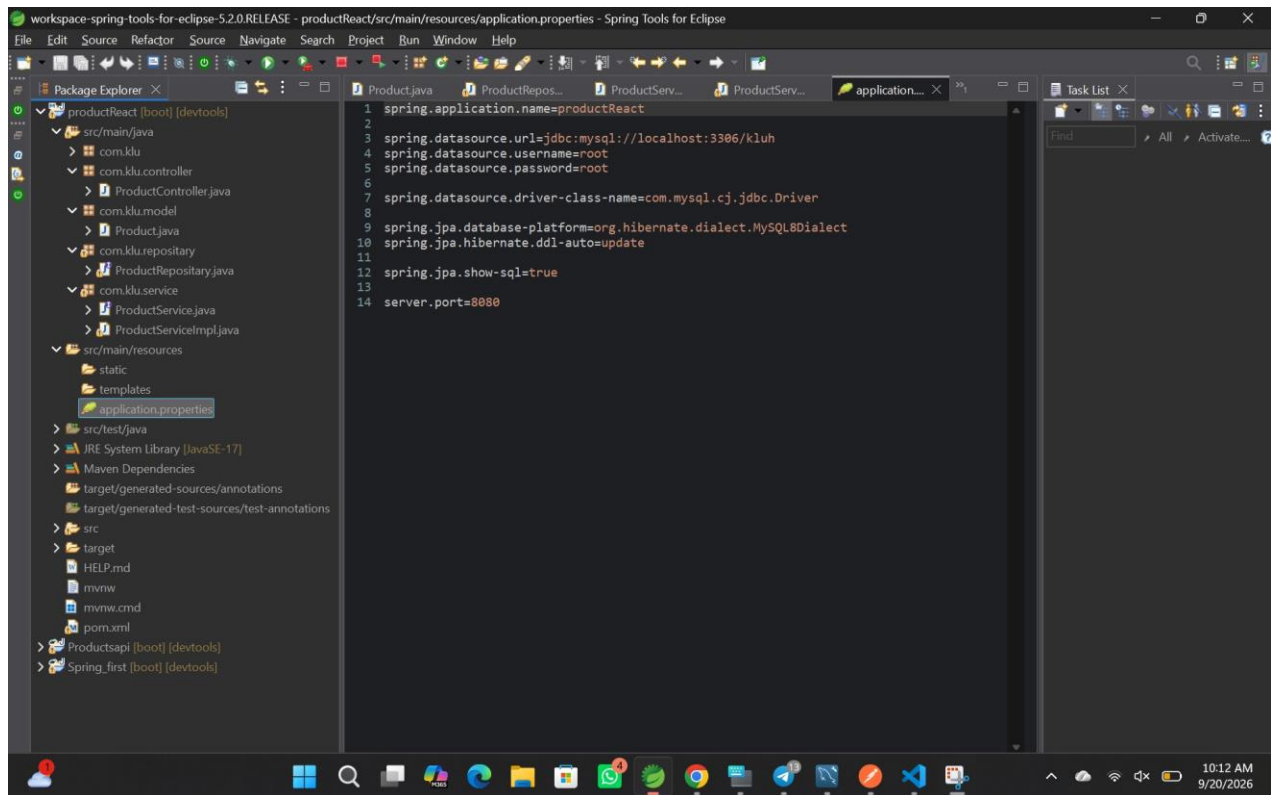
The screenshot shows the Eclipse IDE with the 'ProductServiceImpl.java' file open. The Package Explorer on the left is the same as the previous screenshot. The main editor displays the following code:

```
1
2
3 import java.util.List;
4
5
6 @Service
7 public class ProductServiceImpl implements ProductService {
8
9     private final ProductRepository productRepository;
10
11     @Autowired
12     public ProductServiceImpl(ProductRepository productRepository) {
13         this.productRepository = productRepository;
14     }
15
16     @Override
17     public List<Product> getAllProducts() {
18         return productRepository.findAll();
19     }
20
21     @Override
22     public Product getProductById(Long id) {
23         return productRepository.findById(id).orElse(null);
24     }
25
26     @Override
27     public Product saveProduct(Product product) {
28         return productRepository.save(product);
29     }
30
31     @Override
32     public Product updateProduct(Long id, Product product) {
33         if (productRepository.existsById(id)) {
34             product.setId(id);
35             return productRepository.save(product);
36         }
37         return null;
38     }
39
40     @Override
41     public void deleteProduct(Long id) {
42         productRepository.deleteById(id);
43     }
44 }
```

The Task List on the right is empty. The status bar at the bottom shows the time as 10:11 AM on 9/20/2026.

REACTJS + SPRING BOOT CRUD

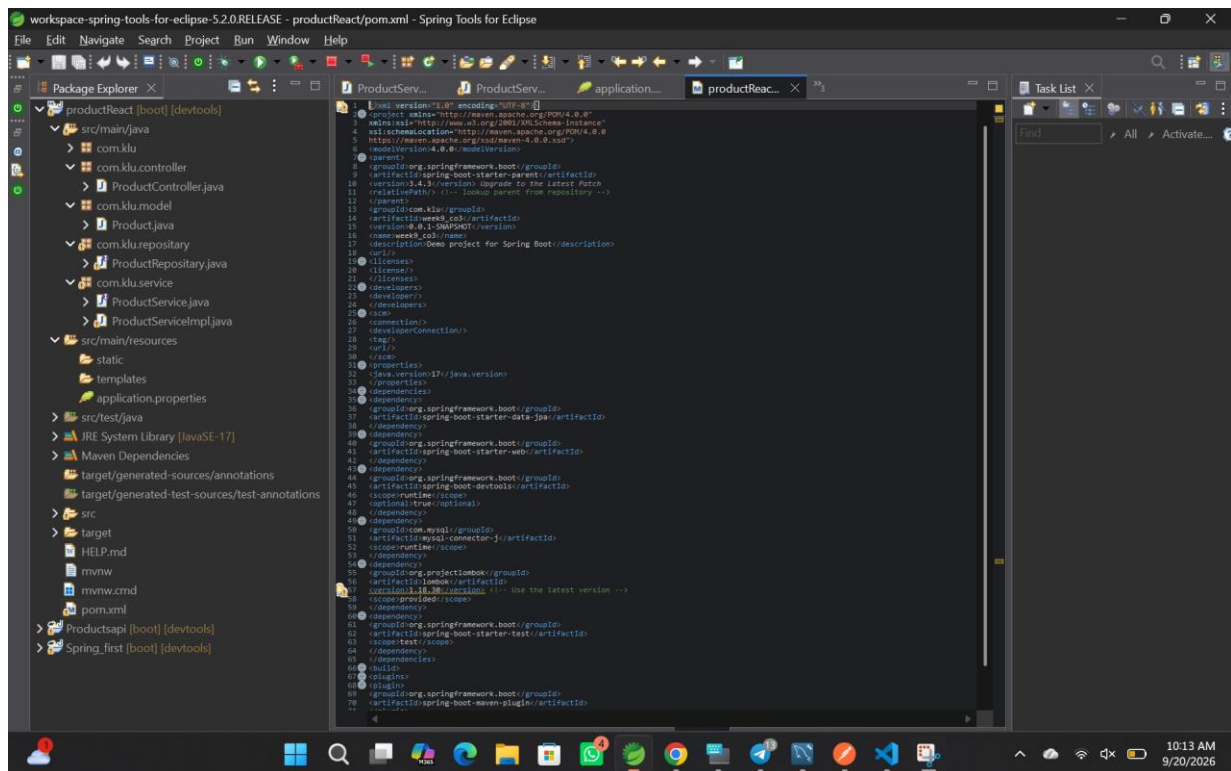
application . properties:



The screenshot shows the Eclipse IDE interface with the 'application.properties' file open in the editor. The Package Explorer on the left shows the project structure for 'productReact'. The editor displays the following properties:

```
1 spring.application.name=productReact
2
3 spring.datasource.url=jdbc:mysql://localhost:3306/k1uh
4 spring.datasource.username=root
5 spring.datasource.password=root
6
7 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
8
9 spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
10 spring.jpa.hibernate.ddl-auto=update
11
12 spring.jpa.show-sql=true
13
14 server.port=8080
```

pom.xml:



The screenshot shows the Eclipse IDE interface with the 'pom.xml' file open in the editor. The Package Explorer on the left shows the project structure for 'productReact'. The editor displays the following XML content:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.4.3</version>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.3</version>
    <relativePath>../..</relativePath>
  </parent>
  <description>Demo project for Spring Boot</description>
  <url>http://spring.io</url>
  <licenses>
    <license>
      <name>Spring Framework</name>
      <url>http://www.springframework.org</url>
    </license>
  </licenses>
  <developers>
    <developer>
      <name>Spring Framework</name>
      <email>springframework@spring.io</email>
      <organization>Spring Framework</organization>
    </developer>
  </developers>
  <scm>
    <connection>scm:git:https://github.com:spring-projects/spring-boot.git</connection>
    <developerConnection>scm:git:https://github.com:spring-projects/spring-boot.git</developerConnection>
    <tag>spring-boot-3.4.3</tag>
  </scm>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
      <scope>compile</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
      <scope>compile</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <scope>compile</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <scope>provided</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```


REACTJS + SPRING BOOT CRUD

OUTPUT:

